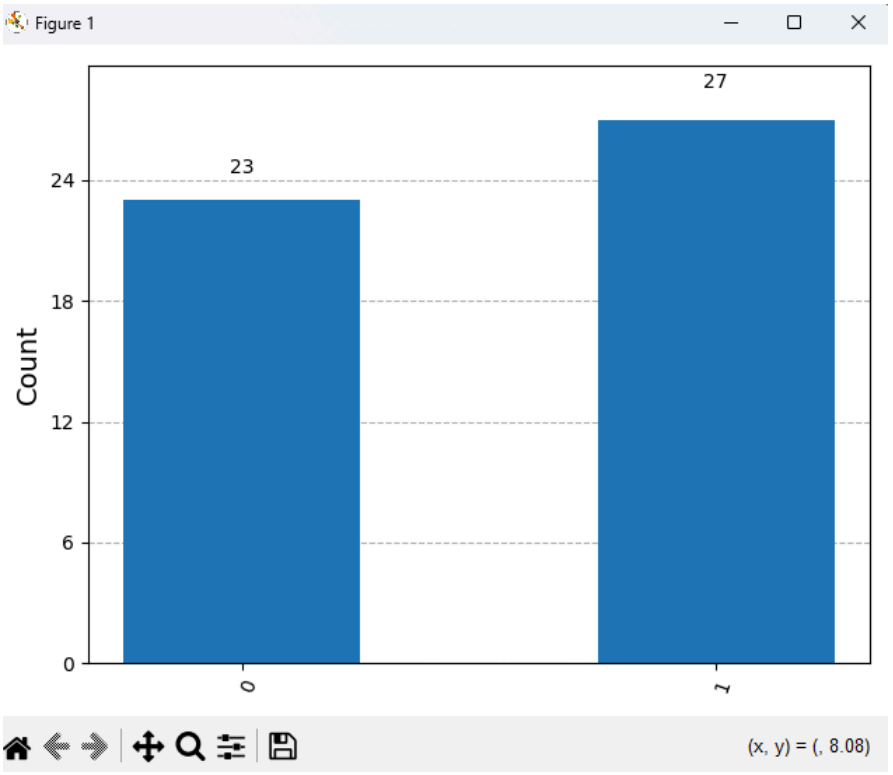
Task 2

Here is the code in my file hello_quantum.py:

```python
from qiskit import QuantumCircuit
from qiskit_aer import Aer
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
qc = QuantumCircuit(1, 1)
qc.h(0)
qc.measure(0, 0)
simulator = Aer.get_backend('aer_simulator')
result = simulator.run(qc, shots=50).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
plt.show()
```

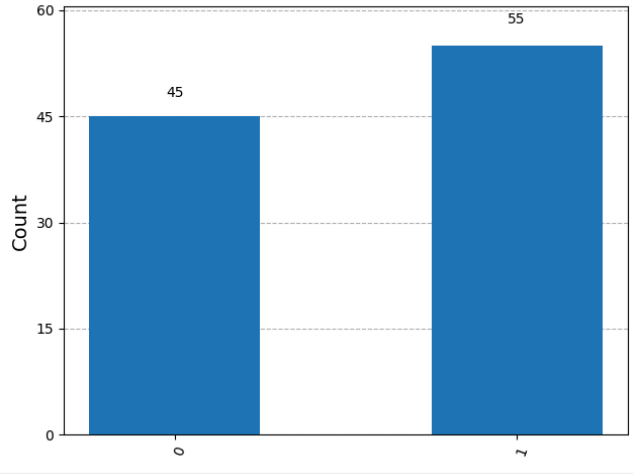
This is the output after running:



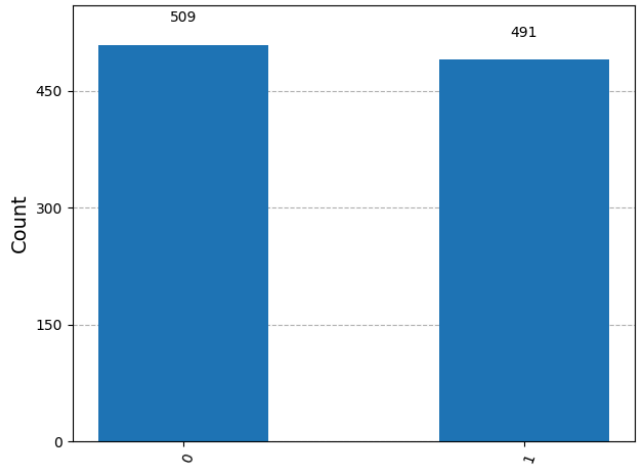
The code creates one qubit and sets it to have an equal chance of being a 0 or 1 when measured. The code also creates a normal bit to store the measured result of the qubit.

We sample the qubit 50 times in the given code to observe the probability distribution. We see a roughly 50/50 split, but not exactly. The qubit was sampled 23 times as a 0 and 27 times as a 1.

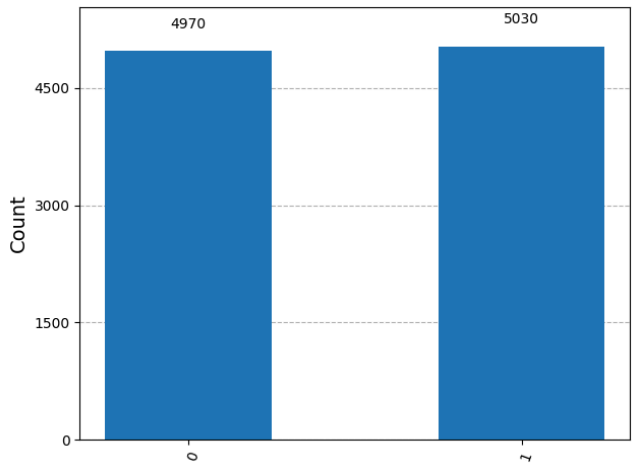
100 shots:



1000 shots:



10000 shots:



As we increase the number of shots, the random fluctuations average out and the measured probabilities approach the true expected probabilities (the law of large numbers in action).

We need multiple runs because one shot only gives one random result, it might be 0 or 1. In quantum programs, to be able to estimate the quantum probabilities, we need to repeat the measurement many times.

Task 3

Here is the code in my file hadamard_test.py:

```python
from qiskit import QuantumCircuit
from qiskit_aer import Aer
qc = QuantumCircuit(1, 1)
# Try without Hadamard first
# qc.h(0)
qc.measure(0, 0)
simulator = Aer.get_backend('aer_simulator')
result = simulator.run(qc, shots=5000).result()
print("Without Hadamard:", result.get_counts())
```

I just uncommented the qc.h(0) line when I wanted to test with the gate enabled.

Here I have copy pasted the printed output instead of screenshotting:

Running 1000 shots without Hadamard: Without Hadamard: {'0': 1000}
Then enabling it and running again: With Hadamard: {'0': 486, '1': 514}

Running 10 shots without Hadamard: Without Hadamard: {'0': 10}
Then enabling it and running again: With Hadamard: {'1': 3, '0': 7}

Running 100 shots without Hadamard: Without Hadamard: {'0': 100}
Then enabling it and running again: With Hadamard: {'1': 58, '0': 42}

Running 5000 shots without Hadamard: Without Hadamard: {'0': 5000}
Then enabling it and running again: With Hadamard: {'0': 2497, '1': 2503}

For small runs, randomness has a stronger effect. The probabilities are still 50/50 when we enable the Hadamard gate, but statistical variation in a small sample can make the distribution uneven, as is seen in the 10 shot case.

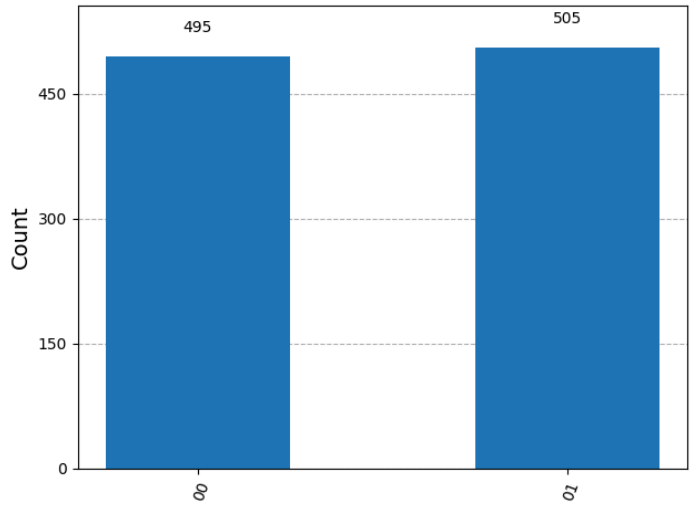
But when we increase the number of samples (as we saw in task 2 above), the measured distributions approach the theoretical 50/50 distribution, because the random fluctuations average out.

Task 4

Here is the default code in entanglement.py, before we introduce entanglement.

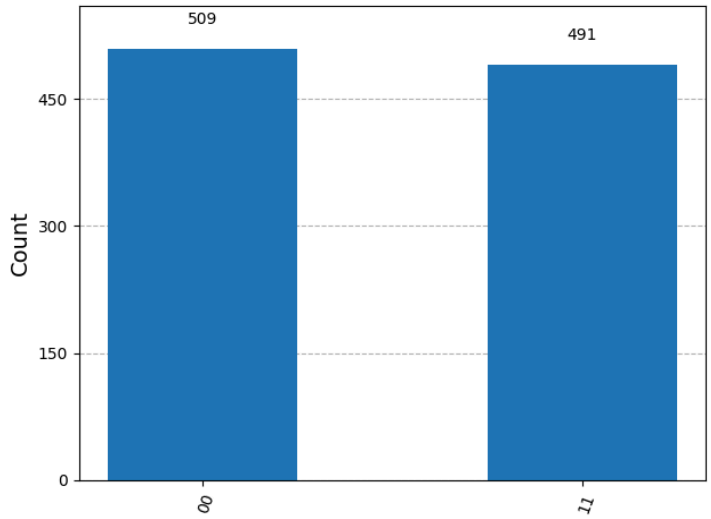
```python
from qiskit import QuantumCircuit
from qiskit_aer import Aer
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.measure([0,1], [0,1])
simulator = Aer.get_backend('aer_simulator')
result = simulator.run(qc, shots=1000).result()
counts = result.get_counts()
print("Without CNOT:", counts)
plot_histogram(counts)
plt.show()
```

This is the output of this code:



This code creates 2 qubits, but only enables a Hadamard gate for one of them. So one has a 50/50 chance of being a 0 or 1 when measured, but the other is always a 0. So we observe an almost 50/50 measured split between two cases, 00 and 01.

Next I enabled a cnot by adding qc.cx(0, 1). This means that one qubit is the control qubit and the other is the target. If the control qubit is observed as a 1, then the target qubit also flips to 1.



This is the result after running the code with the cnot enabled. You can see we still have a roughly 50/50 split between two states, but instead of the states being 00/01 they are now 00/11 because the state of the control qubit affects the state of the target qubit. This is supposed to represent that they are entangled.

Only 00 and 11 can appear precisely because they are entangled, the measurements are perfectly correlated. If qubit 0 is measured as 0, then so is qubit 1. If qubit 0 is measured as 1, then so is qubit 1.

This is not possible classically because classical bits always have definite values. Quantum entanglement creates correlations that cannot be simply described with classical bits. When there is entanglement, the system acts like one combined quantum state rather than two separate quantum bits.

Extension Experiments (copy pasted terminal outputs):

Remove Hadamard, but keep cnot:
No Hadamard, with CNOT: {'00': 1000}

Remove both:
No Hadamard, without CNOT: {'00': 1000}

These both lead to the same outcome. If we remove the Hadamard gate, the control bit stays as 0. In the first case with the cnot enabled, they are entangled, so the other bit stays the same as the control bit. In the second case they are not entangled but because there is no Hadamard gate, both bits just stay as 0.

Changing order of gates:
CNOT first, then Hadamard: {'00': 504, '01': 496}

Notice that doing the gates in this order does NOT create the entangled state that we saw the other way around. I think this is because these quantum gates are not commutative, and the order matters. The cnot gate can only create entanglement if the control bit is already

superimposed, which it is not in this case because we had not yet created the Hadamard gate.

Increasing the number of shots only allows the observed probability distribution to approach the theoretical through the law of large numbers as we saw in previous parts.

Task 5: Reflection

1. Why do quantum programs require multiple executions?
   Quantum measurements are inherently probabilistic. If we only take one measurement we only get one random outcome, but if we want to observe the probability distribution we need to take many repeated measurements.
2. How is this different from classical programs?
   Classical programs are usually deterministic. When you run them they give the same result each time, for a given input. Quantum programs on the other hand might give different outputs due to quantum superposition of qubits and their various measurement probabilities.
3. What does the Hadamard gate achieve?
   The Hadamard gate creates a superposition. It places a qubit into a state where it has a 50/50 chance of existing as a 0 or a 1 when it is observed.
4. What is entanglement in your own words?
   Entanglement is when the measurement of two qubits is perfectly correlated. The measurement of one instantly determines the state of the other, even though their existence before measurement is not known.